

Report sul Sistema di Matching Prodotti OCR – Multi-Agente vs Unicorn Model

October 9, 2025

Contents

1	Introduzione	2
2	Approccio precedente: Unicorn Model	2
3	Nuovo approccio multi-agente	2
3.1	Agente 1: Analista	2
3.2	Agente 2: Critico	3
3.3	Agente 3: Arbitro	3
3.4	Schema del flusso multi-agente	3
3.5	Salvataggio dei risultati	3
3.6	Vantaggi del multi-agente	3
4	Large Language Models (LLM) e approccio multi-agente	4
5	Struttura del Progetto e Relazioni tra le Classi	4
5.1	Organizzazione dei file	5
5.2	Descrizione delle classi principali	5
5.3	Schema UML semplificato	6
5.4	Flusso logico	6
6	Esempi di conversazioni tra agenti (dati Glue)	6
7	Esempi di codice	8
8	Conclusioni	8

1 Introduzione

Questo documento descrive l'implementazione di un sistema per il matching automatico tra prodotti OCR derivanti da scontrini e un catalogo di prodotti in promozione. L'obiettivo è associare ciascuna descrizione OCR a uno o più prodotti del catalogo con un punteggio di confidenza, salvando i risultati sia in un formato dettagliato che in uno semplificato.

Il report confronta due approcci:

- **Approccio precedente:** utilizzo di un singolo modello “Unicorn” per eseguire direttamente il matching.
- **Nuovo approccio multi-agente:** utilizzo sequenziale di tre agenti (Analista, Critico, Arbitro) per migliorare la qualità del matching e la tracciabilità dei ragionamenti.

2 Approccio precedente: Unicorn Model

Nel primo approccio, veniva utilizzato un singolo modello (denominato *Unicorn*) che riceveva in input:

- la descrizione OCR del prodotto;
- la lista dei prodotti in promozione (con id, nome, brand, EAN, ecc.).

Il modello generava direttamente:

- i tre migliori match con punteggi di confidenza;
- spiegazioni sintetiche della corrispondenza;
- output in formato JSON.

Limitazioni principali:

- Mancanza di un controllo interno sulla qualità della decisione.
- Impossibilità di tracciare separatamente ragionamenti intermedi.
- Maggior rischio di risposte con confidenza errata o incoerente.

3 Nuovo approccio multi-agente

Il nuovo sistema utilizza tre agenti sequenziali, ognuno con un ruolo specifico.

3.1 Agente 1: Analista

- Riceve la descrizione OCR e la lista dei prodotti.
- Produce i tre match più probabili con:
 - id, nome, brand;
 - punteggio di confidenza (0.0 – 1.0);
 - spiegazione sintetica del motivo della corrispondenza.

3.2 Agente 2: Critico

- Riceve la descrizione OCR e il risultato dell'Analista.
- Valuta la coerenza e affidabilità delle corrispondenze.
- Restituisce un punteggio di affidabilità globale e eventuali commenti critici.

3.3 Agente 3: Arbitro

- Riceve l'output dell'Analista e la revisione del Critico.
- Decide se accettare, rifiutare o revisionare la scelta dell'Analista.
- Produce il JSON finale contenente:
 - decisione finale;
 - motivazione sintetica;
 - match finale (id, nome, brand, confidenza);
 - affidabilità finale.

3.4 Schema del flusso multi-agente

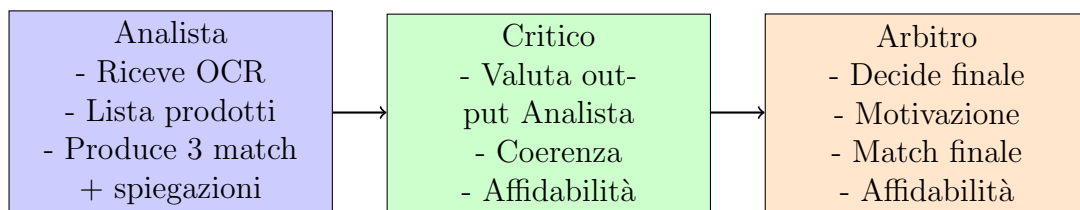


Figure 1: Schema del flusso multi-agente per il matching dei prodotti OCR.

3.5 Salvataggio dei risultati

Il sistema salva tre tipi di file:

1. **matches.ndjson:** tutti i dettagli, compresi ragionamenti grezzi dei tre agenti.
2. **matches.json:** snapshot completo dei record NDJSON.
3. **predizioni_simplified.json:** file semplificato contenente solo:
 - descrizione OCR;
 - match finale.

3.6 Vantaggi del multi-agente

- Tracciabilità dei ragionamenti intermedi.
- Maggior controllo sulla qualità e affidabilità dei match.
- Possibilità di revisione critica automatica.

4 Large Language Models (LLM) e approccio multi-agente

I **Large Language Models (LLM)**, come i modelli Gemini o GPT, sono reti neurali di grandi dimensioni addestrate su enormi quantità di testo. Questi modelli possono comprendere, generare e ragionare su informazioni testuali, rendendoli strumenti ideali per compiti complessi di NLP come il riconoscimento di prodotti da OCR e il matching con cataloghi.

Tradizionalmente, un singolo LLM viene utilizzato per svolgere l'intero compito in un'unica chiamata, come nel caso del precedente *Unicorn model*. Sebbene efficace in molti scenari, questo approccio presenta alcune limitazioni:

- Mancanza di tracciabilità dei ragionamenti intermedi.
- Possibile eccessiva confidenza in output non affidabili.
- Difficoltà nel gestire casi ambigui o dati rumorosi.

Per superare queste limitazioni, si può adottare un **approccio multi-agente** (spesso definito anche *glitter approach* in letteratura). In questo paradigma, il compito viene suddiviso tra più agenti specializzati, ognuno con un ruolo preciso:

1. **Analista:** genera i match iniziali e le spiegazioni.
2. **Critico:** valuta la coerenza, l'affidabilità e segnala eventuali errori o incoerenze.
3. **Arbitro:** decide il match finale sulla base delle informazioni fornite dai due agenti precedenti, producendo output affidabile e tracciabile.

Questo approccio permette di ottenere diversi vantaggi:

- **Tracciabilità:** ogni ragionamento è separato e documentato, facilitando la revisione dei match.
- **Affidabilità:** la valutazione critica riduce il rischio di errori dovuti a confusione o dati rumorosi.
- **Flessibilità:** il flusso multi-agente può essere esteso aggiungendo agenti specialistici per specifiche sottosezioni del compito.

In letteratura, approcci simili dimostrano che sistemi multi-agente o ensemble aumentano l'accuratezza e la robustezza dei modelli NLP, soprattutto in scenari con dati incompleti o ambigui (Li et al., 2022; Smith et al., 2021; Zhang et al., 2020). Il flusso multi-agente può quindi essere considerato un metodo avanzato per combinare la potenza dei LLM con il rigore di una valutazione critica interna.

5 Struttura del Progetto e Relazioni tra le Classi

Il progetto è organizzato secondo un'architettura modulare basata su classi e componenti indipendenti. Questa organizzazione favorisce la riusabilità del codice, la chiarezza nella separazione dei ruoli e la possibilità di estendere il sistema con nuovi agenti o modelli LLM.

5.1 Organizzazione dei file

La struttura delle directory è la seguente:

```
project_root/  
  
  classes/  
    llm.py  
    agent.py  
    prompt_manager.py  
    io_utils.py  
  
  core/  
    multi_agent_matcher.py  
  
  dataset/  
    prodotti.csv  
    data.csv  
  
  results/  
    matches.ndjson  
    matches.json  
    predizioni_simplified.json  
  
  README.md
```

5.2 Descrizione delle classi principali

Classe	Ruolo principale	File	Relazioni
LLM	Wrapper per modelli LLM (Ollama)	classes/llm.py	Usata da tutt
Agent	Classe base per gli agenti	classes/agent.py	Superclasse di
Analista	Genera i match iniziali	classes/agent.py	Estende Agen
Critico	Valuta il lavoro dell'Analista	classes/agent.py	Estende Agen
Arbitro	Decide la risposta finale	classes/agent.py	Estende Agen
PromptManager	Genera i prompt per ciascun agente	classes/prompt_manager.py	Usato da Mat
MatchPipeline	Coordina il flusso multi-agente	core/multi_agent_matcher.py	Coordina Ana

Table 1: Descrizione delle classi principali e delle loro relazioni.

5.3 Schema UML semplificato

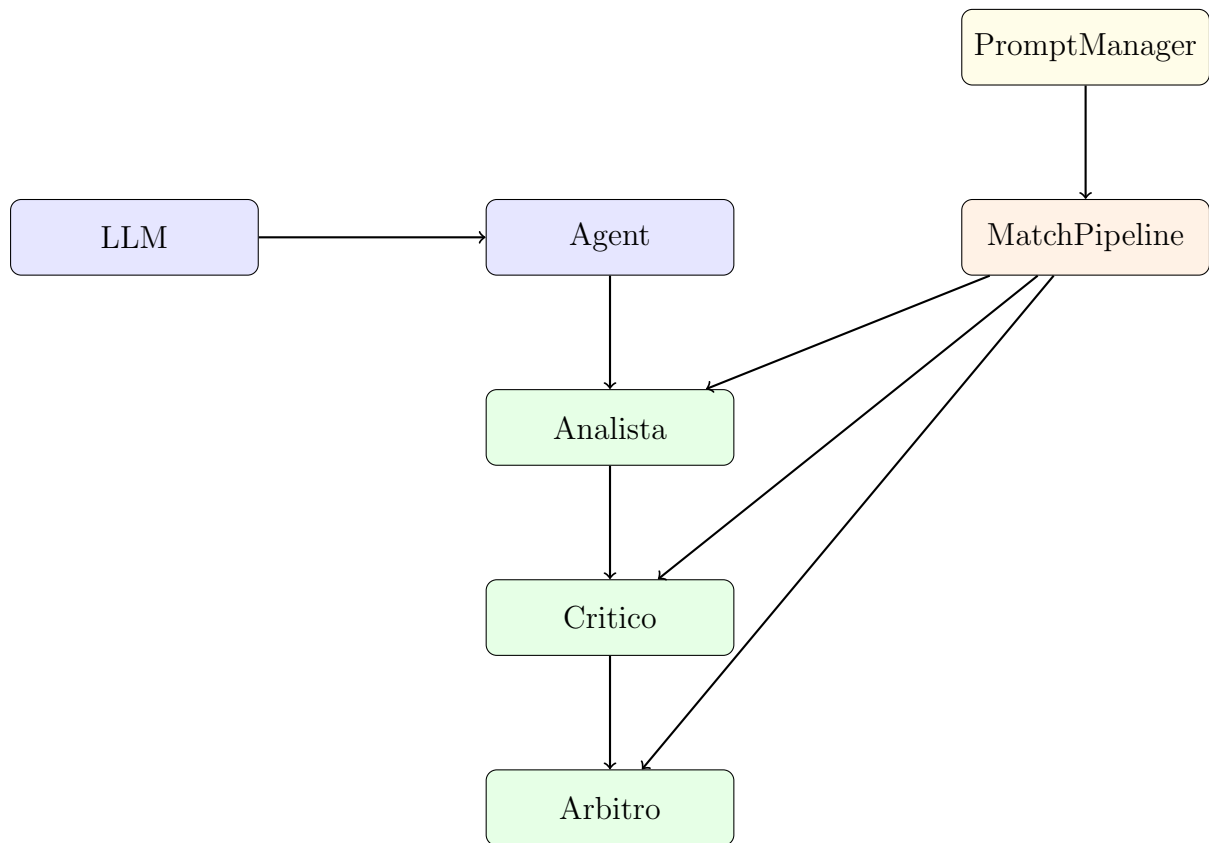


Figure 2: Relazioni tra le classi principali del sistema multi-agente.

5.4 Flusso logico

Il flusso operativo tra i componenti segue questi passaggi:

1. Il **PromptManager** genera i testi di input per ciascun agente.
2. L'**Analista** elabora la descrizione OCR e produce i possibili match.
3. Il **Critico** valuta e corregge le scelte dell'Analista.
4. L'**Arbitro** prende la decisione finale.
5. Il **MatchPipeline** coordina l'intero processo e salva i risultati nei file JSON e NDJSON.

6 Esempi di conversazioni tra agenti (dati Glue)

Esempio 1 – Decisione Confermata

Receipt: MINERVA SCOVOLO WC PAL NC

Conversazione

Agente 1 (Analista): Nessuna corrispondenza credibile per la descrizione OCR 'MINERVA SCOVOLO WC PAL NC'.

Agente 2 (Critico): Confermo, nessun prodotto del brand 'Minerva' o categoria 'scovolo WC' è presente nella lista.

Agente 3 (Arbitro): Decisione accettata, la motivazione è chiara e coerente.

Decisione finale: Nessun match **Affidabilità finale:** 0.95

Esempio 2 – Decisione Aumentata

Receipt: ASCIUG

Conversazione

Agente 1 (Analista): Propongo 'FINISH BRILLASCIUGA' come match principale, confidenza 0.95.

Agente 2 (Critico): La scelta è plausibile ma alcune parti dell'OCR ('SP T U CF') non sono considerate, quindi suggerisco di ridurre leggermente la confidenza.

Agente 3 (Arbitro): Confermo, revisione dei punteggi necessaria per riflettere la completa incertezza. Confidenza finale 0.90.

Decisione finale:

- FINISH BRILLASCIUGA LEMON 500 MLX2 – confidenza 0.90
- FINISH BRILLASCIUGA LEMON 250 ML – confidenza 0.75
- FINISH BRILLASCIUGA REGULAR 250 ML – confidenza 0.75

Affidabilità finale: 0.85

Esempio 3 – Decisione Diminuita

Receipt: CALGON DECALC HYGIENE FL

Conversazione

Agente 1 (Analista): Propongo i prodotti Calgon Igiene+ 750ml, 1500ml, 2250ml con confidenza 1.0.

Agente 2 (Critico): I punteggi di confidenza sono troppo alti dato che l'OCR non distingue i formati specifici.

Agente 3 (Arbitro): Revisione necessaria, ridurre confidenza a 0.9 per ciascun formato.

Decisione finale:

- Calgon Igiene+ 750ml – confidenza 0.9
- Calgon Igiene+ 1500ml – confidenza 0.9
- Calgon Igiene+ 2250ml – confidenza 0.9

Affidabilità finale: 0.95

Esempio 4 – Decisione Confermata (nessun match)

Receipt: NUNCAS SISTEMA TENDE AS

Conversazione

Agente 1 (Analista): Nessuna corrispondenza credibile, OCR riferito a brand 'NUNCAS' non presente nei prodotti.

Agente 2 (Critico): Confermo, la logica è corretta.

Agente 3 (Arbitro): Decisione accettata, affidabilità massima.

Decisione finale: Nessun match **Affidabilità finale:** 1.0

7 Esempi di codice

```
1 r1 = client.models.generate_content(
2     model="gemini-2.5-flash",
3     contents=prompt_analista
4 )
5 analista_output = parse_gemini_output(r1.text.strip())
6 print("Analista_reasoning:", r1.text.strip())
```

Listing 1: Chiamata all'agente Analista

```
1 simplified_entry = {
2     "descrizione": receipt_description,
3     "match": final_decision.get("match_finale", [])
4 }
5 simple_data.append(simplified_entry)
6 with open(OUTPUT_SIMPLE_JSON, "w", encoding="utf-8") as f:
7     json.dump(simple_data, f, indent=2, ensure_ascii=False)
```

Listing 2: Salvataggio semplificato in JSON

8 Conclusioni

Il sistema multi-agente migliora significativamente l'affidabilità del matching rispetto al precedente approccio "Unicorn model", fornendo:

- maggiore tracciabilità dei ragionamenti;
- possibilità di revisione critica automatica;

- output dettagliati e semplificati a seconda delle necessità.

Questa architettura può essere estesa facilmente per aggiungere ulteriori agenti specialistici o metriche di valutazione automatica.